

Riga Technical University
Faculty of Computer Science and Information Technology
Department of Systems Theory and Design

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE

Lecture 8

FRAMES AND SCRIPTS

Dr.habil.sc.ing., professor Janis Grundspenkis, Dr.sc.ing., lecturer Alla Anohina-Naumeca

Department of Systems Theory and Design

Faculty of Computer Science and Information Technology

Riga Technical University

E-mail: {janis.grundspenkis, alla.anohina-naumeca}@rtu.lv

Address: Meza street 1/4- {550, 545}, Riga, Latvia, LV-1048

Phone: (+371) 67089{581, 595}

Frames: idea

Author: Minsky, 1975

Idea: When one encounters a new situation (or makes a substantial change in one's view of a problem) one selects from memory a structure called a “frame”. This is a remembered framework to be adapted to fit reality by changing details as necessary.

According to Minsky a frame is a static data structure used to represent well-understood, stereotyped situations. Framelike structures seem to organize our own knowledge of the world. We adjust to ever new situations by calling up information structured by past experience. We then specially fashion or revise the details of these past experience to represent the individual differences for the new situation.

Frames: structure

Frames represent knowledge as structured objects which is composed of labeled slots with their values.

Slots are similar to attributes in object-oriented approach, however, they can contain broader information not only declarative, but also procedural information.

Frames: slots (1)

Slots can contain the following information:

- Frame identification information (name)

 **Example:**

A frame which stores knowledge about cars can have a name “Car”

- Relationship of this frame to other frames

 **Example:**

A super-class of a frame “Car” is a frame “Vehicle”

- Knowledge about an attribute of an object and its value

 **Example:**

A frame “Car” can have an attribute “Number of wheels” with value 4

- Frame default information (These are slot values that are taken to be true when no evidence to the contrary has been found)

 **Example:**

A frame “Car” can have a slot “Number of doors” with value 4, however, there are cars with only 2 doors

Frames: slots (2)

Slots can contain the following information (continued):

- Unspecified values (Many frame slots may be left unspecified until given a value for a particular instance or needed for problem solving). In this case value of a slot is empty.

Example:

A frame “Car” can contain a slot “Color”, however, this slot will be empty, because there is a lot of different colors. When knowledge about a particular car, for example, John’s car, will be represented, then the slot “Color” will have a certain value.

- Procedural information

Frames: slots (3)

A frame can include:

- *Slots with static values*, values don't change during the operation of a system
- *Slots with dynamic values*, values change during the operation of a systemdarbības laikā

Sources of slot values:

- Initialization
- User
- External programs
- Other frames
- Inheritance

Frames: structure

In general a frame has the following structure:

Name of a frame	
Names of slots	Values of slots



Example:

Car	
Engine	
Number of doors	4

Frames: types (1)

Slot values can point out another frame. By relating frames through slot values a frame system can be acquired.

3 types of frames can be found in a frame system:

1. Class frame. Such a frame includes slots describing an attributes of a class of objects. Typically slots of such frames have default information or unspecified values.



Example:

Class: Vehicle	
Reg.No.	
Model	
Producer	
Owner	

Frames: types (2)

2. A sub-class frame



Example:

Car	
Class: Vehicle	
Reg.No.	
Model	
Producer	
Owner	
Number of doors	4
Engine	

3. An instance frame



Example:

John's car	
Class: Car	
Reg.No.	LA657
Model	850
Producer	BMW
Owner	John
Number of doors	2
Engine	5.0

Frames: relationships (1)

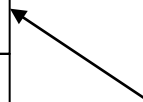
3 types of relationships can relate frames in a frame system:

1. “Is-a” relationship. Relates a sub-class frame with a class frame or an instance frame with a sub-class or class frame. In this case a sub-class frame or an instance frame inherits all slots from a class frame, but it can include also a new slots.



Example:

Class: Vehicle	
Reg.No.	
Model	
Producer	
Owner	



Car	
Class: Vehicle	
Reg.No.	
Model	
Producer	
Owner	
Number of doors	4
Engine	

Inherited slots

New slots

Frames: relationships (2)

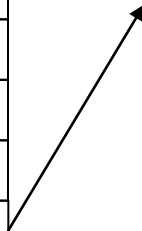
2. “a-part-of” relationship. Relates whole with its constituent parts.



Example:

Truck	
Reg.No.	
Model	
Producer	
Owner	
Part	Basket

Basket	
Dimensions	2*3*1,5



Frames: relationships (3)

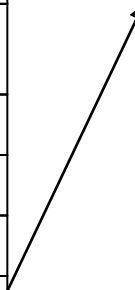
3. Semantic relationship



Example:

John's car	
Class: Car	
Reg.No.	LA657
Producer	BMW
Model	850
Owner	John
Number of doors	2
Engine	5.0

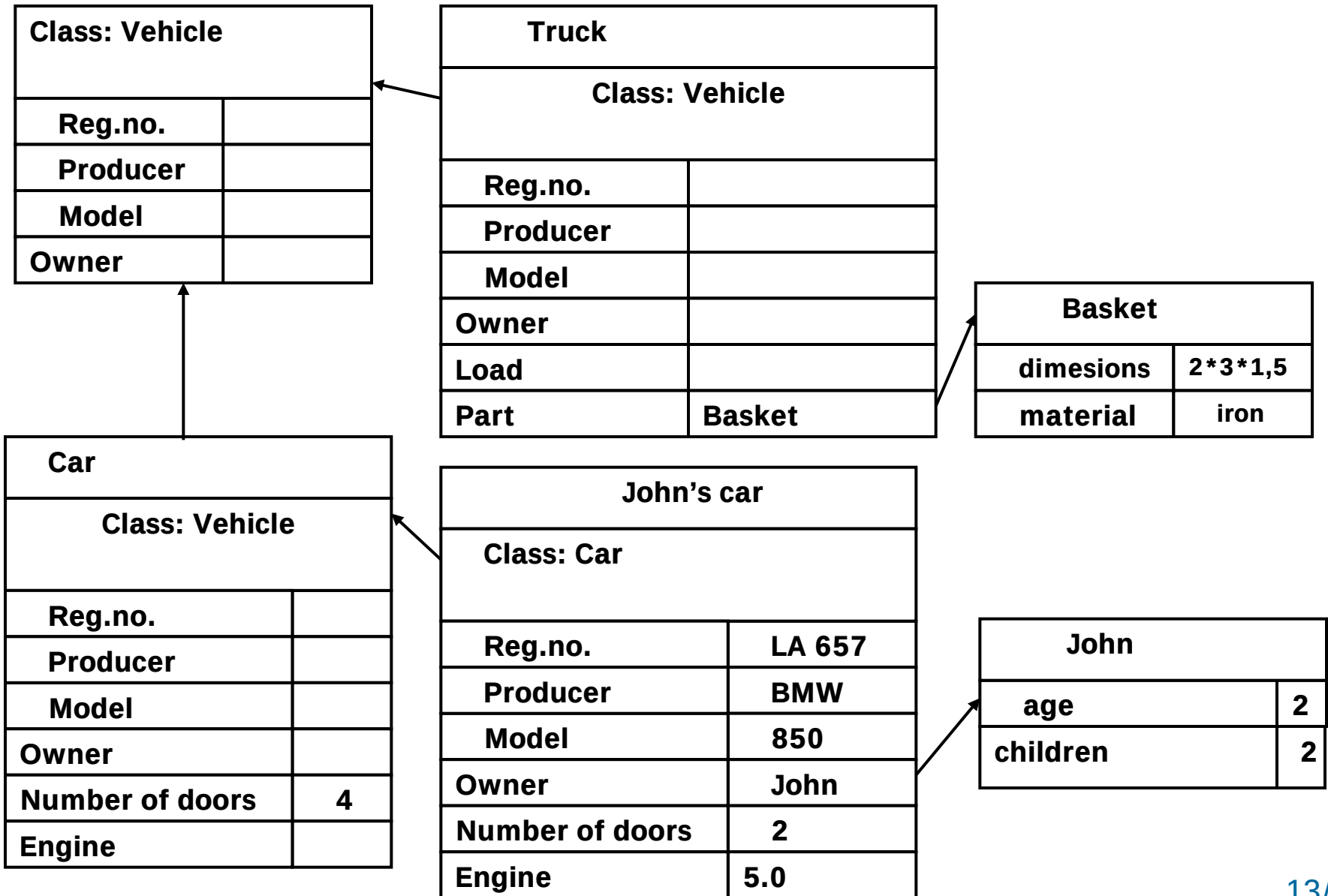
John	
Age	22



Frames: an example



Example:



Frames: facets

Frames can incorporate facets, which represent extended knowledge about slot values. They allow controlling of slot values.

Facets can include:

- Type of a value
- Default value
- Constraints on a value
- Minimum and maximum values
- Actions on values

Name of a frame		
Slots	Slot values	Facets

Frames: methods

Methods called demons can be attached to slots. Demons are invoked automatically when a slot is accessed..

Standard demons are the following:

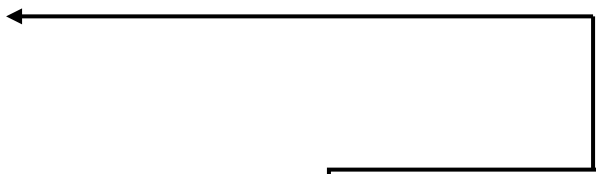
- IF-NEEDED is invoked when it is necessary to acquire a slot value
- IF-CHANGED is invoked when a value of a slot is changed
- IF-ADDED is invoked when a value is added to a slot
- IF-REMOVED is invoked when a value of a slot is deleted

Frames: an example of demons (1)

 **Example:**

Man	
Married	False
Wife	
Age	

if-changed { if Married=true then acquire(Wife) }



John	
class	Man
Married	True
Wife	Mary
Age	27

Frames: an example of demons (2)



Example:

Avio-ticket	
Traveller	if-Added { if Traveller: Age<26 then discount=50% else discount=0% }
Discount	

Ticket305	
class	Avioticket
Traveller	John
Discount	50%

John	
Name	„John“
age	24

Scripts: Definition

Authors: Schank and Abelson, 1977

A script is a structured representation describing a stereotyped sequence of events in a particular context.

Scripts are used in natural language understanding systems to organize a knowledge base in terms of the situations that the system should understand.

Scripts: components

A script is composed of following components:

- Entry conditions that must be true for the script to be called.
- Results or facts that are true once the script has terminated.
- Props or the “things” that make up the content of the script.
- Roles are the actions that the individual participants perform.
- Scenes which present temporal aspects of the script.

Scripts: actions

Describing a script a special symbols of actions are used:

Symbol	Meaning	Example
ATRANS	transfer a relationship	<i>give</i>
PTRANS	transfer physical location of an object	<i>go</i>
PROPEL	apply physical force to an object	<i>push</i>
MOVE	move body part by owner	<i>kick</i>
GRASP	grab an object by an actor	<i>grasp</i>
INGEST	ingest an object by an animal	<i>eat</i>
EXPEL	expel from an animal's body	<i>cry</i>
MTRANS	transfer mental information	<i>tell</i>
MBUILD	mentally make new information	<i>decide</i>
CONC	conceptualize or think about an idea	<i>think</i>
SPEAK	produce sound	<i>say</i>
ATTEND	focus sense organ	<i>listen</i>

Scripts: an example

Script Restaurant	Scene 1: Entering P PTRANS P into restaurant P ATTEND eyes to tables P MBUILD where to sit P PTRANS P to table P MOVE P to sitting position	Scene 3: Eating V ATRANS F to O O ATRANS F to P P INGEST F Option: Return to Scene 2 to order more; otherwise, go to Scene 4
Props •Tables •Menu •F = Food •Check •Money	Scene 2: Ordering (Menu on table) O brings menu) P PTRANS menu to P (S asks for menu) S MTRANS signal to O O PTRANS O to table P MTRANS "need menu" to O O PTRANS O to menu O PTRANS O to table O ATRANS menu to P P MTRANS food list to P * P MBUILD choice of F P MTRANS signal to O O PTRANS O to table P MTRANS 'I want F' to O O PTRANS O to V O MTRANS (ATRANS F) to V V MTRANS 'no F' to O O PTRANS O to P O MTRANS 'no F' to P (go back to *) or (go to Scene 4 at no pay path) V DO (prepare F script) to Scene 3	Scene 4: Exiting P MTRANS to O (O ATRANS check to P) O MOVE write check O PTRANS O to P O ATRANS check to P P ATRANS tip to O P PTRANS P to K P ATRANS money to K P PTRANS P to out of restaurant No pay path Schank un Abelson, 1977
Roles •P = Customer •O = Waiter •V = Cook •K = Cashier •S = Owner Entry conditions •P is hungry •P has money Results •P has less money •P is not hungry •P is pleased (optional) •S has more money		